

MEMORIA PRÁCTICA 1

Asignatura: Administración y Gestión de Base de Datos

Miembros: Rafael Petersen y Xuye Qiu

14/10/2025

ÍNDICE

CONTEXTO	2
PARTE 1: Creación y Carga de la BD	2
1.1 Creación de la BD PracABD1	2
1.2 Creación de TableSpaces y DataFiles y scripts de borrado	2
1.3 Creación y Eliminación de Tablas	3
1.4 Creación y Borrado de Claves Primarias y Foráneas	4
1.5 Ejecución de Scripts y Resultados	5
1.6 Carga de Datos a las Tablas mediante el Symbol System	6
1.6.1 Primera Carga	6
1.6.2 Segunda Carga	6
1.6.3 Tercera Carga	7
1.6.4 Cuarta Carga	7
1.6.5 Quinta Carga	8
PARTE 2: Utilización de Índices	11
2.1 Consultas	11
2.2 Comparación de Tiempos y Uso de Índices	12
2.3 Conclusión del Uso de Índices	14
PARTE 3: Interoperabilidad de SGBDs	14
3.1 Conexión a la BD del puerto 3307	14
3.2 Conexión a la BD del puerto 3306	17
Conclusión Final	18

CONTEXTO

En esta práctica se nos plantea el problema de un centro universitario que carece de un departamento de informática para gestionar la forma en la que tratan la información del centro con distintas tecnologías. En concreto, el centro tiene la información dividida en distintas bases de datos y nos piden unificar la información para facilitar su gestión.

En los siguientes apartados se detallan los problemas planteados en esta práctica y las soluciones propuestas por nuestro equipo para resolverlas.

PARTE 1: Creación y Carga de la BD

En este apartado creamos los scripts necesarios para solucionar cada uno de los subapartados planteados en esta primera parte. No obstante, al principio del todo nos conectamos al escritorio virtual, accedemos al símbolo del sistema en modo administrador y escribimos **net start MySQL90** para arrancar mysql manualmente. Una vez iniciado mysql, accedemos a la aplicación MySQL WB, creamos una conexión e introducimos la contraseña **MySQLescritorioUPM**.

1.1 Creación de la BD PracABD1

```
CREATE DATABASE IF NOT EXISTS PracABD1;  
USE PracABD1;
```

1.2 Creación de TableSpaces y DataFiles y scripts de borrado

(Cada TableSpace tiene su DataFile)

```
-- Tablespace para la tabla Personas  
CREATE TABLESPACE TBLS_Personas  
ADD DATAFILE 'DF_Personas.ibd'  
AUTOEXTEND_SIZE= 40M;
```

```
-- Tablespace para la tabla Cursos  
CREATE TABLESPACE TBLS_Cursos  
ADD DATAFILE 'DF_Cursos.ibd'  
autoextend_size= 40M;
```

```
-- Tablespace para la tabla MatriculadosInteresados  
CREATE TABLESPACE TBLS_MatriculadosInteresados
```

```
ADD DATAFILE 'DF_MatriculadosInteresados.ibd'  
AUTOEXTEND_SIZE=40M;
```

```
-- Eliminar tablespaces (incluyendo los datafiles asociados)  
DROP TABLESPACE TBLS_MatriculadosInteresados;  
DROP TABLESPACE TBLS_Cursos;  
DROP TABLESPACE TBLS_Personas;
```

1.3 Creación y Eliminación de Tablas

```
-- Crear tabla Personas en TBLS_Personas  
CREATE TABLE Personas (  
    ID_Persona INT PRIMARY KEY UNIQUE NOT NULL,  
    DNI CHAR(9) NOT NULL UNIQUE,  
    Nombre VARCHAR(20) NOT NULL,  
    Apellido VARCHAR(30) NOT NULL,  
    Genero CHAR(1) CHECK (Genero IN ('H', 'M')),  
    Dirección VARCHAR(60),  
    Localidad VARCHAR(50),  
    Provincia VARCHAR(30),  
    CodPostal CHAR(9),  
    Teléfono CHAR(9),  
    EnParo CHAR(1) CHECK (EnParo IN ('0','1')),  
    Canal CHAR(1) CHECK (Canal IN ('0','1', '2','3','4')),  
    FechaNac DATE,  
    Email VARCHAR(60)  
)  
TABLESPACE TBLS_Personas;  
  
-- Crear tabla Cursos en TBLS_Cursos  
CREATE TABLE Cursos (  
    ID_Curso INT PRIMARY KEY UNIQUE NOT NULL,  
    Nombre VARCHAR(15) NOT NULL UNIQUE,  
    Area VARCHAR(30),  
    Edicion INT NOT NULL CHECK (Edicion BETWEEN 2008 AND 2020)  
)  
TABLESPACE TBLS_Cursos;  
  
-- Crear tabla MatriculadosInteresados en TBLS_MatriculadosInteresados  
CREATE TABLE MatriculadosInteresados (  
    ID_Persona INT,  
    ID_Curso INT,  
    Estado VARCHAR(50),  
    Descripcion VARCHAR(500),  
    PRIMARY KEY (ID_Curso, ID_Persona),  
    CONSTRAINT FK_Personas FOREIGN KEY (ID_Persona)  
        REFERENCES Personas(ID_Persona)
```

```

        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_Cursos FOREIGN KEY (ID_Curso)
        REFERENCES Cursos(ID_Curso)
        ON DELETE CASCADE
        ON UPDATE CASCADE
)
TABLESPACE TBLS_MatriculadosInteresados;

```

```

-- Eliminar tablas, pero antes de eliminar una tabla asegurarse de que si su clave primaria
-- es foranea en otra tabla, eliminar la clave foranea en la otra tabla antes
DROP TABLE IF EXISTS MatriculadosInteresados CASCADE;
DROP TABLE IF EXISTS Cursos CASCADE;
DROP TABLE IF EXISTS Personas CASCADE;

```

1.4 Creación y Borrado de Claves Primarias y Foráneas

```

-- Clave primaria sobre PersonalID
ALTER TABLE Personas
ADD CONSTRAINT ID_Personas PRIMARY KEY (ID_Persona);

-- Clave primaria sobre CursoID
ALTER TABLE Cursos
ADD CONSTRAINT ID_Cursos PRIMARY KEY (ID_Curso);

-- Clave primaria compuesta sobre ID_MatriculadosInteresados
ALTER TABLE MatriculadosInteresados
ADD CONSTRAINT ID_MatriculadosInteresados PRIMARY KEY (ID_Persona, ID_Curso);

-- MatriculadosInteresados → Definir Clave foránea Personas
ALTER TABLE MatriculadosInteresados
ADD CONSTRAINT FK_Personas FOREIGN KEY (ID_Persona)
    REFERENCES Personas(ID_Persona)
    ON DELETE CASCADE
    ON UPDATE CASCADE;

-- MatriculadosInteresados → Definir Clave Foránea Cursos
ALTER TABLE MatriculadosInteresados
ADD CONSTRAINT FK_Cursos FOREIGN KEY (ID_Curso)
    REFERENCES Cursos(ID_Curso)
    ON DELETE CASCADE
    ON UPDATE CASCADE;

-- Eliminar FK de CURSO
ALTER TABLE MatriculadosInteresados

```

```
DROP FOREIGN KEY FK_Cursos;
```

```
-- Eliminar PK compuesta de MatriculadosInteresados
```

```
ALTER TABLE MatriculadosInteresados
```

```
DROP PRIMARY KEY;
```

```
-- Eliminar FK de PERSONA
```

```
ALTER TABLE MatriculadosInteresados
```

```
DROP FOREIGN KEY FK_Personas;
```

```
-- Eliminar PK de Personas
```

```
ALTER TABLE Personas
```

```
DROP PRIMARY KEY;
```

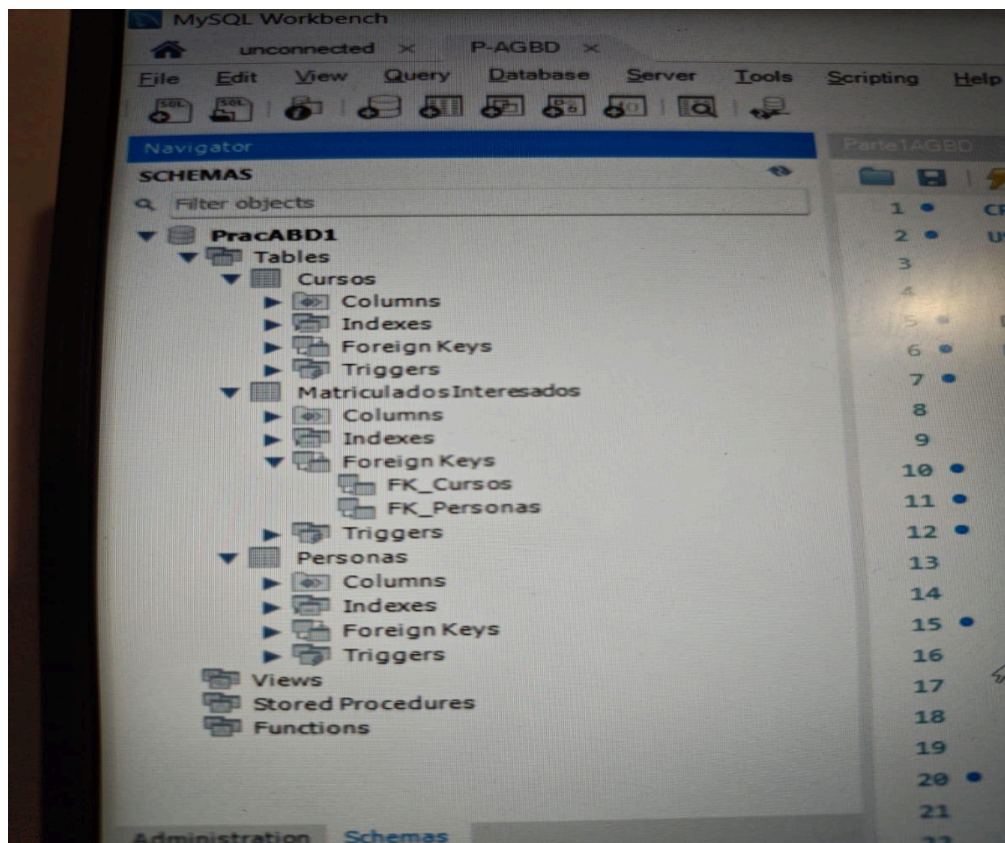
```
-- Eliminar PK de Cursos
```

```
ALTER TABLE Cursos
```

```
DROP PRIMARY KEY;
```

1.5 Ejecución de Scripts y Resultados

A continuación hay una imagen de la base de datos resultante tras aplicar los scripts de creación de los subapartados anteriores



1.6 Carga de Datos a las Tablas mediante el Symbol System

En este subapartado nos dan 3 hojas de excel con los datos que tenemos que cargar a las tablas correspondientes de la base de datos, pero antes de cargarlos hay que exportarlos como ficheros csv. Se nos presentan dos formas de cargar datos, nosotros hemos elegido cargarlos desde el símbolo del sistema.

Primero insertamos estas instrucciones para conectarnos a la base de datos y decirle que queremos cargar datos:

```
"C:\Program Files\MySQL\MySQL Workbench 8.0 CE\mysql.exe" -u root -p --local-infile=1  
use nombreBD;  
set global local_infile=1;
```

1.6.1 Primera Carga

Posteriormente introducimos la dirección del archivo csv y la tabla de la base de datos que queremos cargar. Primero comenzamos cargando la tabla Personas, así que introducimos:

```
LOAD DATA LOCAL INFILE 'C:/Users/Usuario/Downloads/Personas.csv'  
INTO TABLE Personas  
CHARACTER SET utf8mb4  
FIELDS TERMINATED BY ','  
LINES TERMINATED BY '\n';
```

Tras la carga nos aparecen más de 3000 warnings, y nos damos cuenta de que de las 82032 filas a cargar solo ha cargado 30000, por lo que es necesario saber qué ha pasado. Para ello tecleamos en el símbolo del sistema: show warnings;

Como primer warning tenemos: **Warning | 1265 | Data truncated for column 'FechaNac' at row 1**, y vemos que este warning aparece más de una vez y si nos vamos a la tabla personas para comprobar sus valores nos podemos dar cuenta de que la columna fecha no tiene valores, esto se debe a que estamos intentando asociar un string a una dato de tipo DATE.

1.6.2 Segunda Carga

Para solucionarlo hay que hacer la conversión, así que borramos la anterior tabla Personas y cargamos de nuevo con una nueva función:

```
LOAD DATA LOCAL INFILE 'C:/Users/Usuario/Downloads/Personas.csv'  
INTO TABLE Personas  
CHARACTER SET utf8mb4  
FIELDS TERMINATED BY ','  
LINES TERMINATED BY '\n'  
(ID_Persona, DNI, Nombre, Apellido, Genero, Dirección, Localidad, Provincia, CodPostal,  
Teléfono, EnParo, Canal, @fechaProvisional, Email)
```

SET

```
FechaNac = IF(@fechaProvisional = "", NULL, STR_TO_DATE(@fechaProvisional, '%d/%m/%Y'));
```

La modificación añadida cambia el valor de fecha actual que está leyendo del .csv por NULL si no tiene valor o lo cambia por un valor del tipo Date con el formato día/mes/año si tiene valor. Tras esta carga, siguen apareciendo warnings, pero menos que antes y observamos en la tabla personas que la columna fecha ya tiene valores válidos.

Seguidamente, le damos de nuevo a 'show warnings;' para ver qué más errores hay. Esta vez como primer warning aparece: **Warning | 3819 | Check constraint 'Personas_chk_2' is violated.** Esto indica que nuestro segundo check en la tabla personas está dando problemas, vamos a ver de qué trata.

1.6.3 Tercera Carga

En la tabla Personas tenemos la segunda restricción check en el atributo EnParo, esto implica que hay filas en la columna EnParo en el fichero .csv con valor distinto a 0 o 1. Para saber que más valores aparte de los permitidos están tomando las filas de personas en la columna EnParo, accedemos a excel y aplicamos un filtro a la columna EnParo y nos damos cuenta de que hay filas que toman el valor vacíos. Para solucionarlo modificamos la carga de datos incluyendo unas líneas que conviertan los valores vacíos en 1 en la columna EnParo:

```
LOAD DATA LOCAL INFILE 'C:/Users/Usuario/Downloads/Personas.csv'
INTO TABLE Personas
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ';'
LINES TERMINATED BY '\n'
(ID_Persona, DNI, Nombre, Apellido, Genero, Dirección, Localidad, Provincia, CodPostal,
Teléfono, @EnParo, Canal, @fechaProvisional, Email)
SET
  FechaNac = IF(@fechaProvisional = "", NULL, STR_TO_DATE(@fechaProvisional, '%d/%m/%Y')),
  EnParo = IF(@EnParo = "", 1, @EnParo);
```

Tras cargar los datos observamos una vez más que hay menos warnings que antes, pero siguen habiendo. Una vez más escribimos en Symbol System 'show warnings' y nos aparece los siguiente: **Warning | 3819 | Check constraint 'Personas_chk_3' is violated.**

1.6.4 Cuarta Carga

Este warning ya no ha aparecido antes, así que no es desconocido. Para solucionarlo, accedemos al script de creación de la tabla Personas y vemos que el tercer check pertenece al atributo Canal. Para saber que posibles valores puede estar teniendo canal más allá de los definidos, nos metemos a la tabla de personas en excel y aplicamos un filtro

a la columna correspondiente a la columna Canal y encontramos que también contiene valores vacíos. De modo que la solución para este apartado es prácticamente idéntica al la anterior solución pero aplica al atributo Canal:

```
LOAD DATA LOCAL INFILE 'C:/Users/Usuario/Downloads/Personas.csv'
INTO TABLE Personas
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ';'
LINES TERMINATED BY '\n'
(ID_Persona, DNI, Nombre, Apellido, Genero, Dirección, Localidad, Provincia, CodPostal,
Teléfono, @EnParo, @Canal, @fechaProvisional, Email)
SET
  FechaNac = IF(@fechaProvisional = "", NULL, STR_TO_DATE(@fechaProvisional,
'%d/%m/%Y')),
  EnParo = IF(@EnParo = "", 1, @EnParo),
  Canal= NULLIF(@Canal,"");
```

Tras esta carga observamos que se han cargado con éxito toda la información del fichero .csv porque se han cargado con éxito las 87032 filas sin ningún warning, pero tras revisar en la tabla resultante en workbench nos damos cuenta de que la columna email permite acentos y ñ, pero en el enunciado nos pide borrar acentos y sustituir ñ por n.

1.6.5 Quinta Carga

Para solucionar el problema anterior basta con poner una nueva función aplicada a email como se aprecia en el siguiente código:

[illegible]

```

REPLACE(
  REPLACE(
    REPLACE(@Email, 'á', 'a'),
    'é', 'e'),
    'í', 'i'),
    'ó', 'o'),
    'ú', 'u'),
    'Á', 'A'),
    'É', 'E'),
    'Í', 'I'),
    'Ó', 'O'),
    'Ú', 'U'),
    'ñ', 'n');

```

Tras esta última carga ya hemos conseguido cargar exitosamente el fichero .csv Personas completo y saneado, a continuación se muestran imágenes del resultado en el Símbolo del sistema y del mysql wb:

The screenshot shows a MySQL command window with the following content:

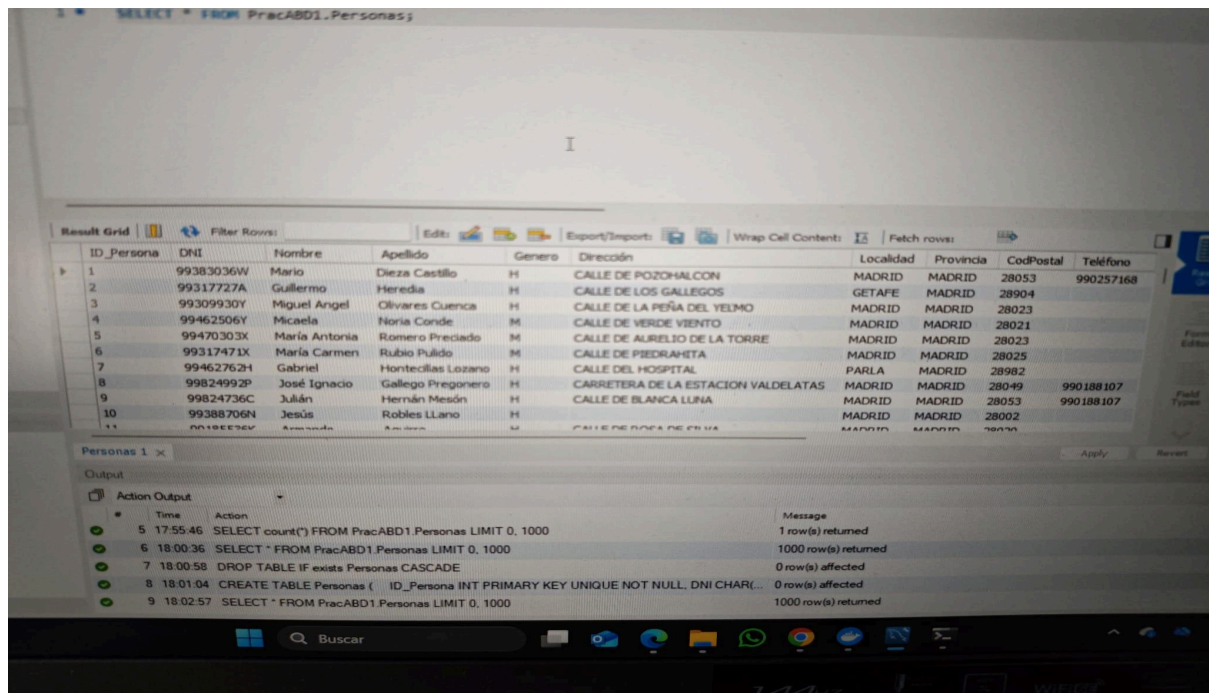
```

Simbolo del sistema - "C:\Pro... x + v
, @fechaProvisional, @Email)
-> SET
-> FechaNac = IF(@fechaProvisional = '', NULL, STR_TO_DATE(@fechaProvisional, '%d/%m/%Y')),
-> EnParo = NULLIF(@EnParo, ''),
-> Canal= NULLIF(@Canal, ''),
-> Email = REPLACE(
->   REPLACE(
->     REPLACE(
->       REPLACE(
->         REPLACE(
->           REPLACE(
->             REPLACE(
->               REPLACE(@Email, 'á', 'a'),
->               'é', 'e'),
->             'í', 'i'),
->           'ó', 'o'),
->         'ú', 'u'),
->       'Á', 'A'),
->     'É', 'E'),
->   'Í', 'I'),
-> 'Ó', 'O'),
-> 'Ú', 'U'),
-> 'ñ', 'n');
Query OK, 87032 rows affected (2.66 sec)
Records: 87032 Deleted: 0 Skipped: 0 Warnings: 0
mysql>

```

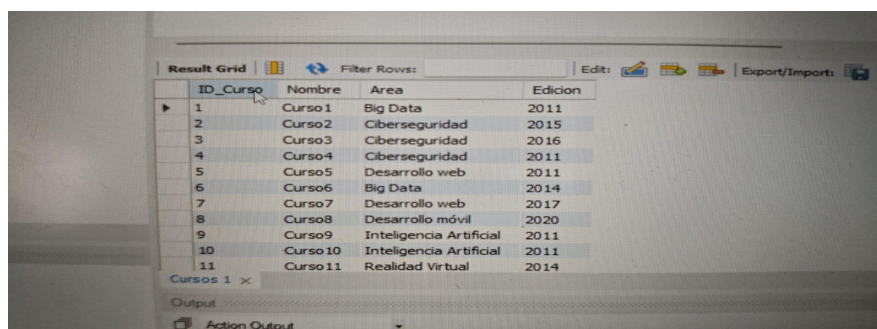
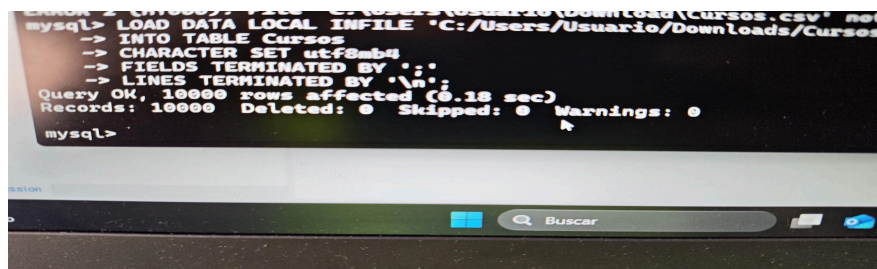
At the bottom of the window, a log shows the following commands and their results:

Time	Command	Result
7 18:00:58	DROP TABLE IF EXISTS Personas CASCADE	0 row(s) affected
8 18:01:04	CREATE TABLE Personas (ID_Persona INT PRIMARY KEY UNIQUE NOT NULL, DNI CHAR(...	0 row(s) affected
9 18:02:57	SELECT * FROM PracABD1.Personas LIMIT 0, 1000	1000 row(s) returned



Una vez cargada el fichero .csv de personas continuamos con la carga del fichero .csv de cursos y matriculados_interesados que son cargas directas y que no suponen problemas, salvo en la tabla matriculados_interesados que aparece un **warning que dice algo parecido a columnas insuficientes o atributos insuficientes o parámetros dados de más**. Para solucionarlo solo hemos tenido que añadir un atributo más en la tabla matriculados_interesados que correspondía a una columna adicional en su correspondiente tabla en excel. A continuación dejamos imágenes de lo que muestra el Símbolo del sistema tras cargar la tabla cursos e Interesados_matriculados y de lo que aparece en workbench.

Cursos:



Matriculados_interesados:

```
-> LINES TERMINATED BY '\n':
Query OK, 10000 rows affected (0.18 sec)
Records: 10000 Deleted: 0 Skipped: 0 Warnings: 0

mysql> LOAD DATA LOCAL INFILE 'C:/Users/Usuario/Downloads/Matr
-> INTO TABLE MatriculadosInteresados
-> CHARACTER SET utf8mb4
-> FIELDS TERMINATED BY ','
-> LINES TERMINATED BY '\n';
Query OK, 113096 rows affected (2.60 sec)
Records: 113096 Deleted: 0 Skipped: 0 Warnings: 0

mysql>
```

ID_Persona	ID_Curso	Estado	Descripción
3477	1		Lorem ipsum dolor sit am
5073	1		Lorem ipsum dolor sit amet, consectetur adipis...
9424	1		Lorem ipsum dolor sit amet, consectetur adipis...
18056	1		Lorem ipsum dolor sit am
24436	1		Lorem ipsum dolor sit amet, consectetur adipis...
44178	1	1	Lorem ipsum dolor sit amet, consectetur adipis...
74658	1		Lorem ipsum dolor sit amet, consectetur adipis...
81622	1		Lorem ipsum dolor sit amet, consectetur adipis...
94933	1		
96754	1		Lorem ipsum dolor sit am
99130	1		Lorem ipsum dolor sit amet, consectetur adipis...

PARTE 2: Utilización de Índices

En la segunda parte nos piden realizar varias consultas y tratar de optimizarlas con índices.

2.1 Consultas

a)

```
SELECT Email
FROM Personas
WHERE Nombre = 'Ángeles' AND Apellido = 'del Nido';
```

b)

```
SELECT Email
FROM Personas
WHERE EnParo = 0;
```

c)

```
SELECT COUNT(*) AS NumPotenciales
FROM Personas
WHERE Provincia = 'Lugo';
```

d)

```
SELECT CONCAT(p.Nombre, ' ', p.Apellido) AS NombreCompleto,
       c.Area,
       m.Descripción
FROM MatriculadosInteresados m
```

```

JOIN Personas p ON m.ID_Persona = p.ID_Persona
JOIN Cursos c ON m.ID_Curso = c.ID_Curso
WHERE p.Provincia = 'Madrid' AND c.Edicion = 2018 AND m.Estado = 1
ORDER BY p.Apellido;

```

e)

```

SELECT CONCAT(p.Nombre, ' ', p.Apellido) AS NombreCompleto,
       p.Email
FROM MatriculadosInteresados m
JOIN Personas p ON m.ID_Persona = p.ID_Persona
WHERE m.ID_Curso = 254 AND m.Estado = 1;

```

f) Consulta adicional que cuenta el número de estudiantes actuales por área de curso

```

SELECT c.Area,
       COUNT(*) AS NumAlumnos
FROM MatriculadosInteresados m
JOIN Cursos c ON m.ID_Curso = c.ID_Curso
WHERE m.Estado = 1
GROUP BY c.Area
ORDER BY NumAlumnos DESC;

```

g) Consulta adicional que cuenta el número de estudiantes actuales por provincia

```

SELECT p.Provincia,
       COUNT(*) AS NumAlumnos
FROM MatriculadosInteresados m
JOIN Personas p ON m.ID_Persona = p.ID_Persona
WHERE m.Estado = 1
GROUP BY p.Provincia
ORDER BY NumAlumnos DESC;

```

2.2 Comparación de Tiempos y Uso de Índices

En este subapartado nos piden desactivar las claves primaria y foráneas de las tablas implicadas en las anteriores consultas y ejecutarlas para comparar su tiempo de ejecución con las mismas consultas pero con las claves activadas. En la columna izquierda tenemos el tiempo de ejecución de las consultas con las claves desactivadas y a la derecha con las claves activadas:

- a) 0.047 s / 0.063 s
- b) 0.015 s / 0.016 s
- c) 0,031 s / 0,047 s
- d) 0,219 s / 0,093 s
- e) 0,110 s / 0,015 s
- f) 0,125 s / 0,109 s
- g) 0,171 s / 0,140 s

En los resultados podemos ver que la activación de las claves primarias y foráneas en búsquedas sencillas no mejoran mucho el tiempo de las consultas en comparación con cuando están desactivadas, pero en consultas más complejas sí que las mejoran, debido a que las claves primarias crean automáticamente un índice que evita forzar al gestor de bases de datos a tener que recorrer una tabla entera en busca de un ID concreto.

Pero si quisiéramos mejorar todavía más los tiempos anteriores podríamos utilizar índices como los siguientes:

a)

```
CREATE INDEX idx_nombre_apellidos ON Personas(Nombre, Apellido);
```

b)

```
CREATE INDEX idx_enparo ON Personas(EnParo);
```

c)

```
CREATE INDEX idx_provincia ON Personas(Provincia);
```

d)

```
CREATE INDEX idx_matriculados_persona ON MatriculadosInteresados(ID_Persona);  
CREATE INDEX idx_matriculados_curso ON MatriculadosInteresados(ID_Curso, Estado);  
CREATE INDEX idx_cursos_edicion ON Cursos(Edicion);  
CREATE INDEX idx_personas_prov_apellidos ON Personas(Provincia, Apellido);
```

e)

Reutilizamos índices creados previamente.

```
CREATE INDEX idx_matriculados_curso ON MatriculadosInteresados(ID_Curso, Estado);
```

f)

Reutilizamos índices creados previamente.

```
CREATE INDEX idx_matriculados_curso ON MatriculadosInteresados(ID_Curso, Estado);  
CREATE INDEX idx_persona ON MatriculadosInteresados(ID_Persona);
```

g)

```
CREATE INDEX idx_cursos_id_area ON Cursos(ID_Curso, Area);
```

Reutilizo:

```
CREATE INDEX idx_matriculados_curso ON MatriculadosInteresados(ID_Curso, Estado);
```

Tras activar estos índices obtenemos los nuevos tiempos mostrados en la tercera columna:

- a. 0.047 s / 0.063 s / 0.000 s
- b. 0.015 s / 0.016 s / 0.000 s
- c. 0,031 s / 0,047 s / 0.000 s
- d. 0,219 s / 0,093 s / 0.031 s
- e. 0,110 s / 0,015 s / 0.000 s

- f. 0,125 s / 0,109 s / 0.032 s
- g. 0,171 s / 0,140 s / 0.062 s

Con índices observamos que los tiempos en las consultas se han mejorado, sin embargo no todos los índices creados son muy útiles, como por ejemplo el índice para el atributo EnParo de la tabla Personas, debido a que solo puede tomar dos valores, y al final no es tan selectivo como si fuera un índice en el Dni. Y si muchos valores de paro son iguales, no aportaría el índice mucha mejora en eficiencia. En general, todos los índices creados son útiles en este contexto, porque han mejorado los tiempos de búsqueda de cada consulta, sin embargo, puede que no hayan sido del todo eficientes hablando en términos de memoria ocupada. Por otro lado, si las consultas más empleadas en la base de datos son la a y la d, basta con simplemente crear los índices definidos en esos apartados para mejorar la eficiencia en velocidad de búsqueda en la base de datos en general.

2.3 Conclusión del Uso de Índices

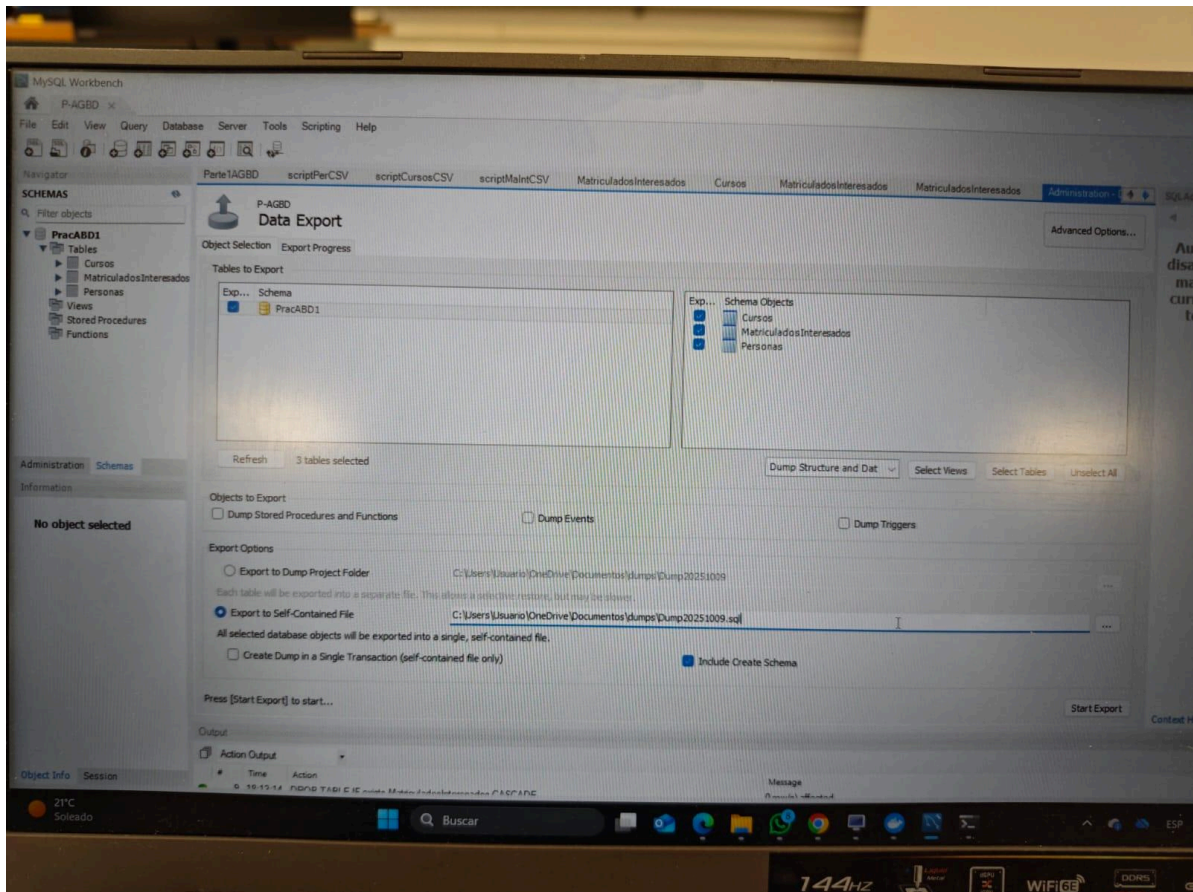
Los índices han demostrado ser fundamentales para mejorar el rendimiento de las consultas más frecuentes y críticas del sistema, especialmente aquellas que implican búsquedas por nombre, provincia, curso o estado de matriculación. Al reducir el número de registros examinados y acelerar los joins, los índices bien diseñados permiten que la base de datos responda de forma mucho más eficiente, cumpliendo con el objetivo de que “vaya como un cohete”. Pero hay que tener cuidado de abusar de los índices ya que también suponen un coste de almacenamiento, así que es imprescindible un estudio exhaustivo de las necesidades de la base de datos para encontrar un equilibrio entre velocidad y coste de almacenamiento.

PARTE 3: Interoperabilidad de SGBDs

En este apartado nos piden utilizar a través de la web gracias a Apache la aplicación phpMyadmin para conectarnos a un servidor MySQL ejecutándose en el puerto 3307, en el que tenemos que crear una base de datos e importar un backup previamente hecho con las tablas de la parte 1 de la práctica. Una vez importada la información desde phpMyadmin realizaremos modificaciones y desde workbench tendremos que comprobar que se han efectuado esas modificaciones creando una conexión a la base de datos del puerto 3307. Después realizaremos lo mismo pero aplicado a la base de datos del puerto 3306.

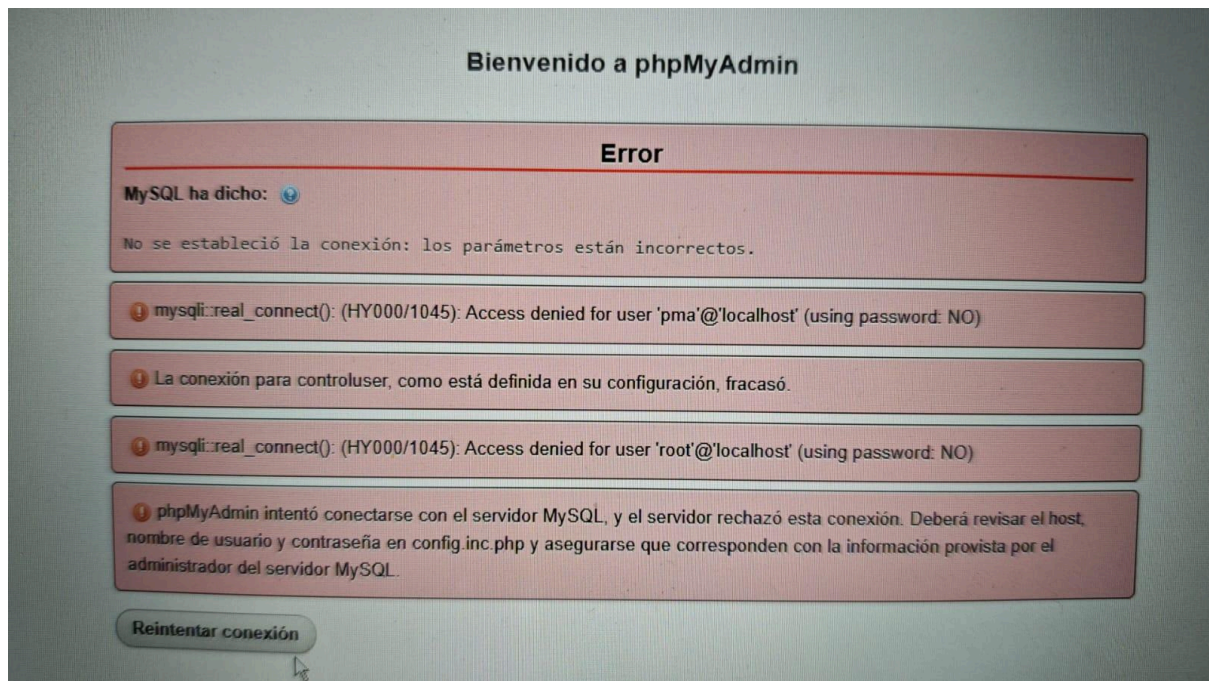
3.1 Conexión a la BD del puerto 3307

Primero vamos a hacer la backup de la parte 1. Para ello tenemos que acceder al Workbench acceder a la conexión donde tenemos la práctica, una vez dentro en orden seleccionamos server>data export ,y cuando se nos abra una ventana seleccionamos lo que aparece en la siguiente imagen:



Posteriormente abrimos Xampp, ejecutamos apache y mySQL probablemente aparezca un error como el siguiente: **Error: MySQL shutdown unexpectedly. This may be due to a blocked port, missing dependencies.** Esto ocurre porque hemos intentado ejecutar el servidor mysql de xampp para que se conecte al puerto 3306, cuando ya había otro servidor mysql conectado a ese puerto que es el instalado en el escritorio virtual. Pero ahora mismo nos interesa conectar el servidor mysql a puerto 3307, así que en el panel de xampp le damos a config de mysql y cambiamos todos los valores 3306 por 3307. Tras este cambio, mysql ya estará conectado al puerto 3307.

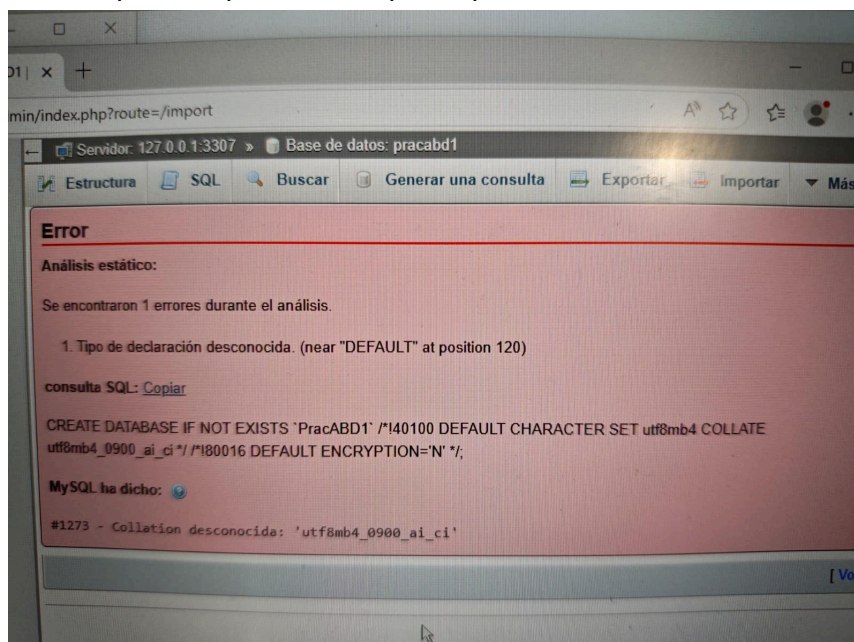
Si ahora le damos a admin en apache estaremos abriendo una pagina web que nos permitirá utilizar la aplicación de phpMyadmin tras pinchar en la sección de phpMyadmin. No obstante, ocurre un error:



Esto ocurre porque no hemos definido el puerto al que se tiene que conectar phpMyadmin, así que desde xampp le damos al config de apache, seleccionamos phpMyAdmin(config) y metemos la siguiente línea debajo de `$cfg['Servers'][$i]['host'] = '127.0.0.1' :`
`$cfg['Servers'][$i]['port'] = '3307';` - - línea a añadir

Tras guardar el documento, le damos de nuevo al admin de apache dentro de la página seleccionamos phpmyAdmin y esta vez ya funcionará la app.

Dentro creamos una nueva BD con el nombre PracABD1 y le damos a importar dejando las demás opciones por defecto, pero aparecerá un error:



Para solucionarla simplemente modificamos el fichero backup sustituyendo utf8mb4_... por utf8mb4_general_ci.

Tras esta modificación importamos de nuevo la backup en la BD y tendremos éxito.

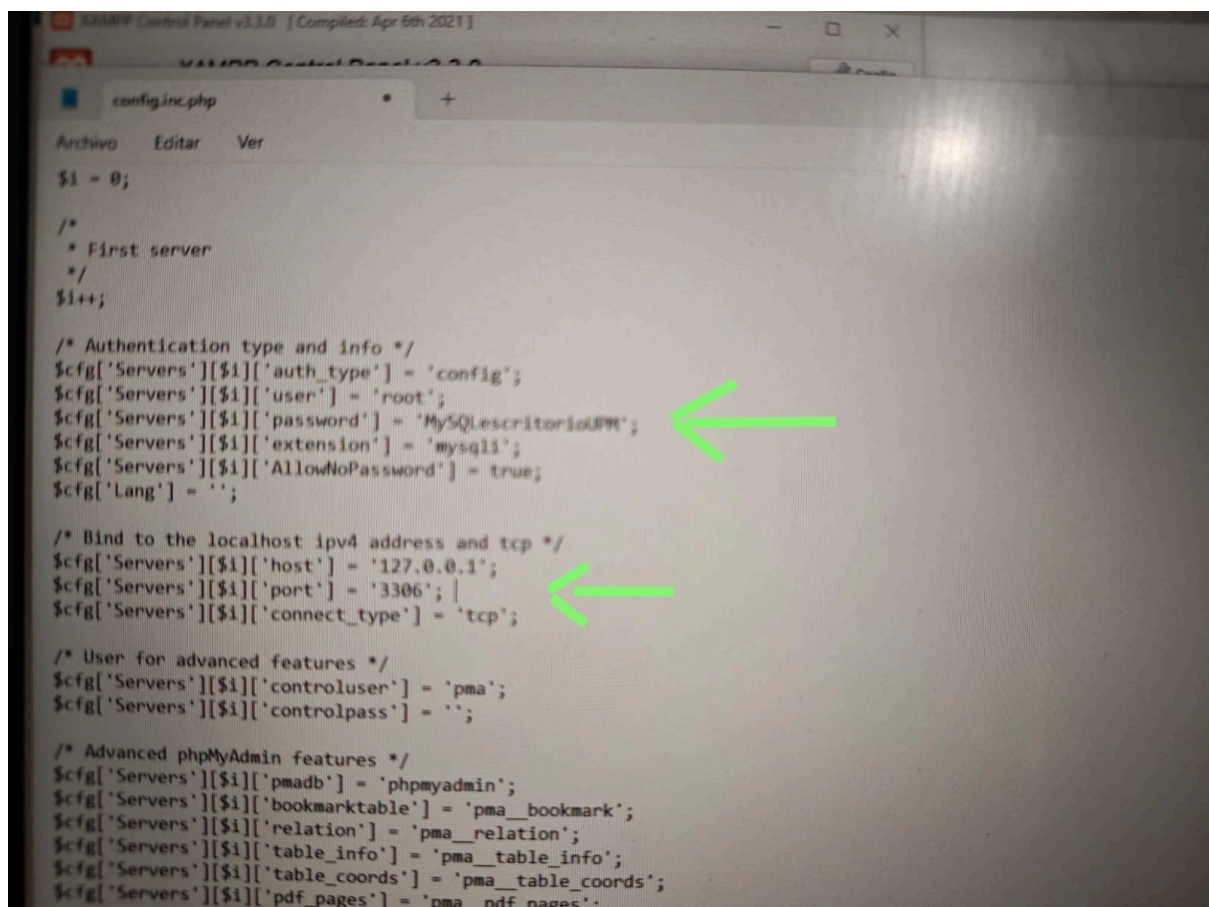
Llegados a este punto ya tenemos phpMyadmin conectado con la base de datos del puerto 3307, ahora queremos conectarnos con workbench a la misma base de datos. Para ello, abrimos workbench y creamos una nueva conexión, ponemos el nombre de la conexión, **el puerto debe ser 3307 y la contraseña vacía**”.

Accedemos a la nueva conexión y ya tenemos workbench conectado a la BD del puerto 3307 y también a phpMyAdmin conectado a la misma BD, ahora los cambios realizados por una parte se verán reflejados en la otra.

3.2 Conexión a la BD del puerto 3306

Ahora vamos a realizar lo mismo de antes pero aplicado al puerto 3306, es decir, conectaremos a workbench y phpMyAdmin a la base de datos del puerto 3306.

Desde MySQL workbench no tenemos que hacer nada más que meternos a la conexión donde tenemos la base de datos del puerto 3306. En Xampp tendremos que cambiar en el config de apache el puerto al que se conectará phpMyAdmin y la contraseña del root de la BD del puerto 3306:



Tras modificar el fichero, ejecutamos admin y en la página emergente pinchamos en phpMyAdmin y ya tendremos acceso a la BD del puerto 3306. En este caso no ha hecho falta iniciar el servidor MySQL desde xampp, porque el servidor mysql del puerto 3306 ya la habíamos iniciado desde el Símbolo del sistema para poder utilizar la conexión desde workbench. Ahora ya podemos modificar los datos de la BD del puerto 3306 ya sea desde Workbench o phpMyAdmin.

Conclusión Final

Esta práctica ha supuesto un recorrido completo por las tareas fundamentales de administración de bases de datos: desde el diseño y creación de tablas, hasta la carga de datos desde ficheros .csv y la conexión entre distintos entornos y puertos de bases de datos. Más allá de los aspectos técnicos, el verdadero valor de esta experiencia ha estado en el autoaprendizaje y la resolución autónoma de problemas, que han sido sin duda, el objetivo principal marcado por el profesor.

Aunque se permitía el uso de herramientas de apoyo como la inteligencia artificial, no ha sido un camino exento de dificultades. En varias ocasiones, la IA ofrecía soluciones que no se ajustaban del todo al contexto específico de la práctica, o resultaba complejo explicarle con precisión el entorno técnico en el que estábamos trabajando (por ejemplo, diferencias entre Workbench y XAMPP, o la lógica de los tablespaces). Esto nos obligó a contrastar, validar y, en muchos casos, corregir o adaptar las soluciones propuestas, especialmente en la carga de ficheros .csv, lo que reforzó aún más nuestra comprensión.